Lesson 1 of 6

Configuration Best Practices

Configuration Goals

Rhapsody can sustain very high message loads on most hardware platforms. Configurations might be simple, composed of only a few routes and communication points, or complex, composed of many routes and communication points. Enterprise-level implementations typically have more than 100 routes and 300 communication points. Very large sites can have configurations of more than 1,000 routes and 3,000 communication points.

Rhapsody frequently deals with patient data, and the goal for the configuration is to enhance (or at least have no negative effect on) patient outcomes. The configuration should be:

- Error free. The configuration should implement the specification correctly and be able to be validated to demonstrate correctness; zero errors is an appropriate goal.
- Maintainable. The purpose and function of all components should be clear and unambiguous.
- Able to ensure that all messages can reach the correct destination in the correct form.
- Able to ensure that messages which cause an exception while processing are identified and processed appropriately.
- Able to ensure that messages are processed through the engine in a timely manner, as required by the system specification.

Best Practice Objectives

The Best Practice objectives define the principles that should be applied during configuration and maintenance of a Rhapsody site. They can be summarized as follows:

- Build route layouts which clearly identify purpose and functionality and are presented in a way that is easy to interpret. Often, the simplest solution is best.
- Ensure the configuration is built to process messages efficiently. This typically means minimizing the number of conditional connectors and filters.
- Establish a clear naming convention for lockers, folders, communication points, filters, and connectors that identify their purpose or function.
- Document all components appropriately using a standard template approach, including author, date and purpose details. A history of all changes should be

included wherever appropriate, along with an indication as to which project or solution this configuration is part of.

- Ensure each component is fully tested before submitting the configuration and that test messages and procedures are included in the configuration.

Route Layout

As we've previously discussed, the route editing canvas presents a flow diagram of the logic required to process messages, with message flow always moving from left to right. A route should include:

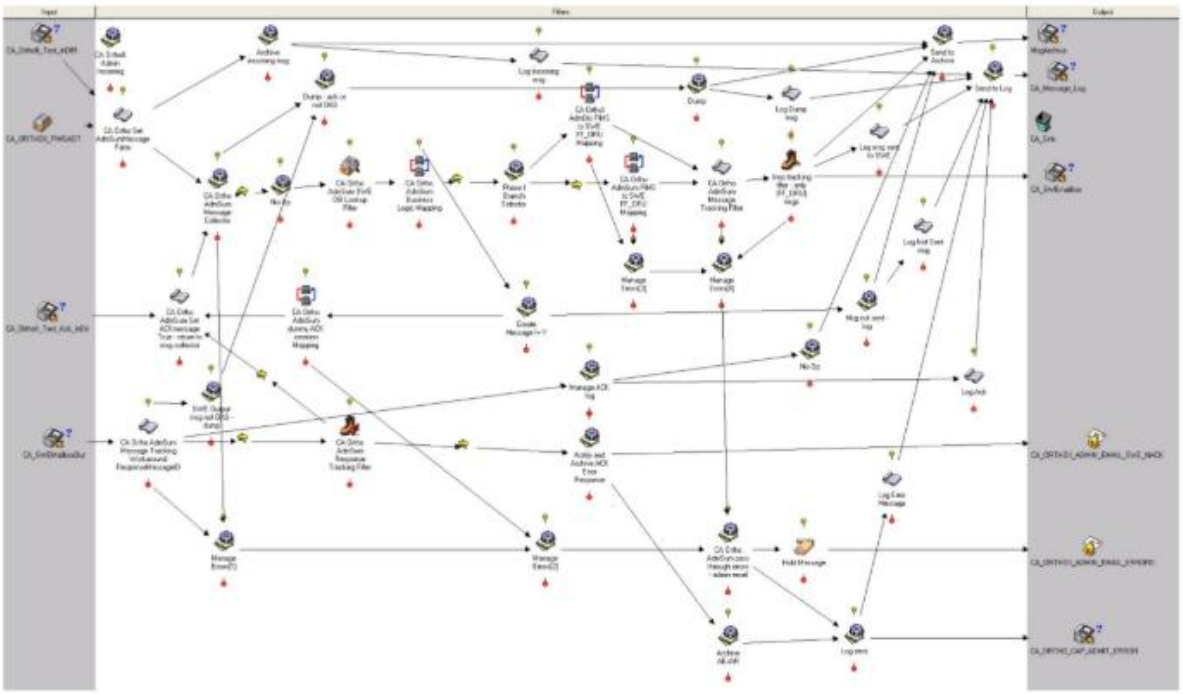
- One or more Input components.
- One or more processing paths consisting of connections and possibly filters.
- One or more Output components.
- Associated message definitions.
- Route properties, including startup state, route processing priority and FIFO handling.
- Route notes that provide an overview of each route and how it is used to process messages.

Points to Consider

When building a route:

- It should be as simple as possible yet still meet all site requirements.
- Keep the number of components to a minimum and ensure that their layout is logical and easy to interpret at-a-glance.
- Connectors should not cross each other and should generally not be aligned vertically, where they may cause confusion with a filter's No-match and Error connectors.

The following image is an example of a poorly configured route. What do you notice about the route layout? How does this impact the best practice objectives?



Fixing a solution such as the one above includes **Route Interconnectivity**, where the output from one route becomes the input for another. This topic will be covered later in this module.

Communication Point Names

As we discussed in the "Building a Simple Route" module, when naming folders and lockers, components should be named to help identify their function and purpose. Activities that benefit from a consistent naming convention include:

- Searching for a component in the Management Console.
- Analyzing and understanding the processing logic.

When developing a naming convention for communication points and filters, consider incorporating some or all the following:

- **Type** and **Direction** flags.
- **Source** or **Target** system name.
- Significant configuration information, such as a **TCP Client** communication point's port number.
- A brief indication of what the component does, such as Map ADT to XML for a **Mapper** filter.

Including all of the above could result in long component names. If a name occupies more than three lines in the route editing canvas, consider shortening it. The use of

spaces in a name rather than underscores is also recommended as it aids in their formatting and readability.

Direction Indicators

Where direction indicators are used in component names, they should be Rhapsody-centric. That is, Input means that Rhapsody is receiving the message, while Output means that Rhapsody is sending the message.

Route Naming

Routes are utilized in the IDE (and Management Console) to provide information on the path taken by a message as it passes through the engine, for example, **PAS ADT Feed** or **PAS to RIS**. You can also number routes to indicate the order they will process messages. For example, the source route would have prefix **1** and the delivery route prefix **2**. This ensures that routes are presented in the workspace in the same order that they process messages.

Filter Names

Filter names should indicate what is happening to the message at this point in the configuration. Filters should never be left with their default name.

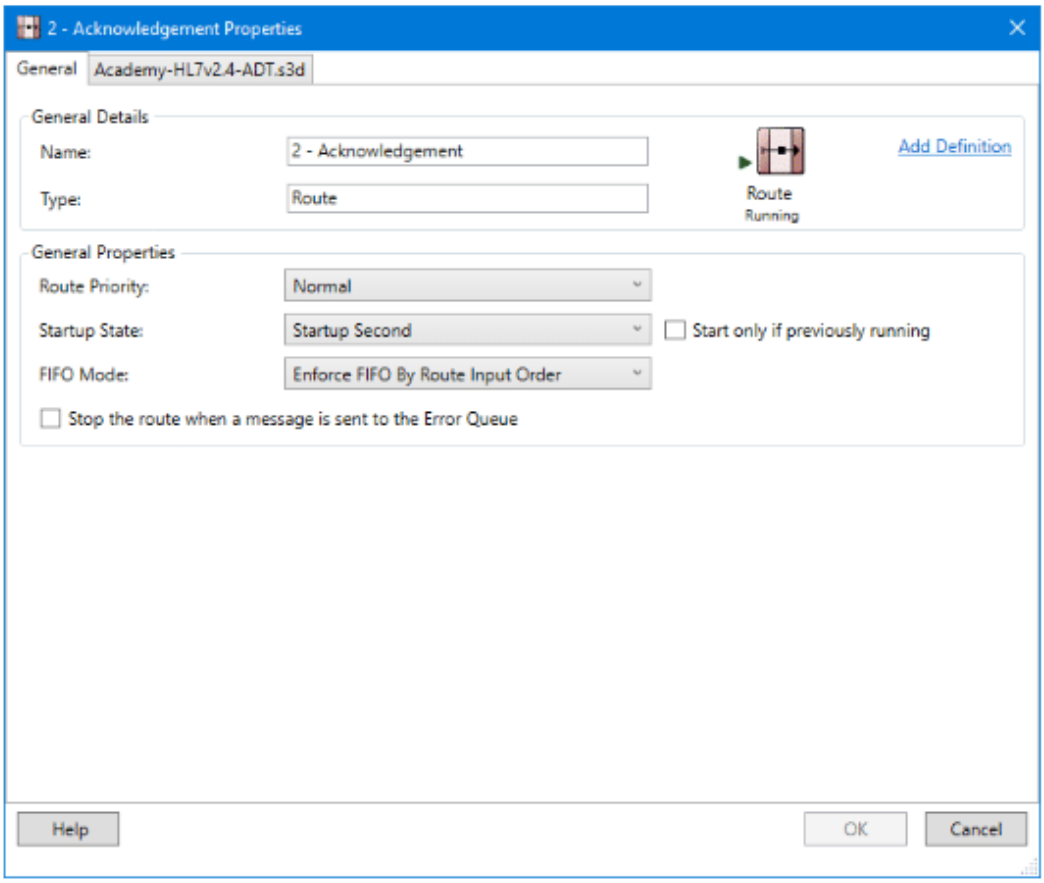
Naming Message Properties

If a message property is used on a route, then ideally that property should be defined on the same route. Although Rhapsody message properties can be passed between routes, this suggested restriction ensures that people can see where the message property came from and can find it easily when needed. Therefore, if the property is not set on the route it is used in, it should be set using a filter that is clearly named. For example, **Set StoreToDatabase Property**.

Lesson 2 of 6

Route Properties

Right-click a route in the **Workspace** to open its **Properties** dialog. The following screenshot shows a route that is currently running with the associated message definition Academy-HL7v2.4-ADT.s3d.

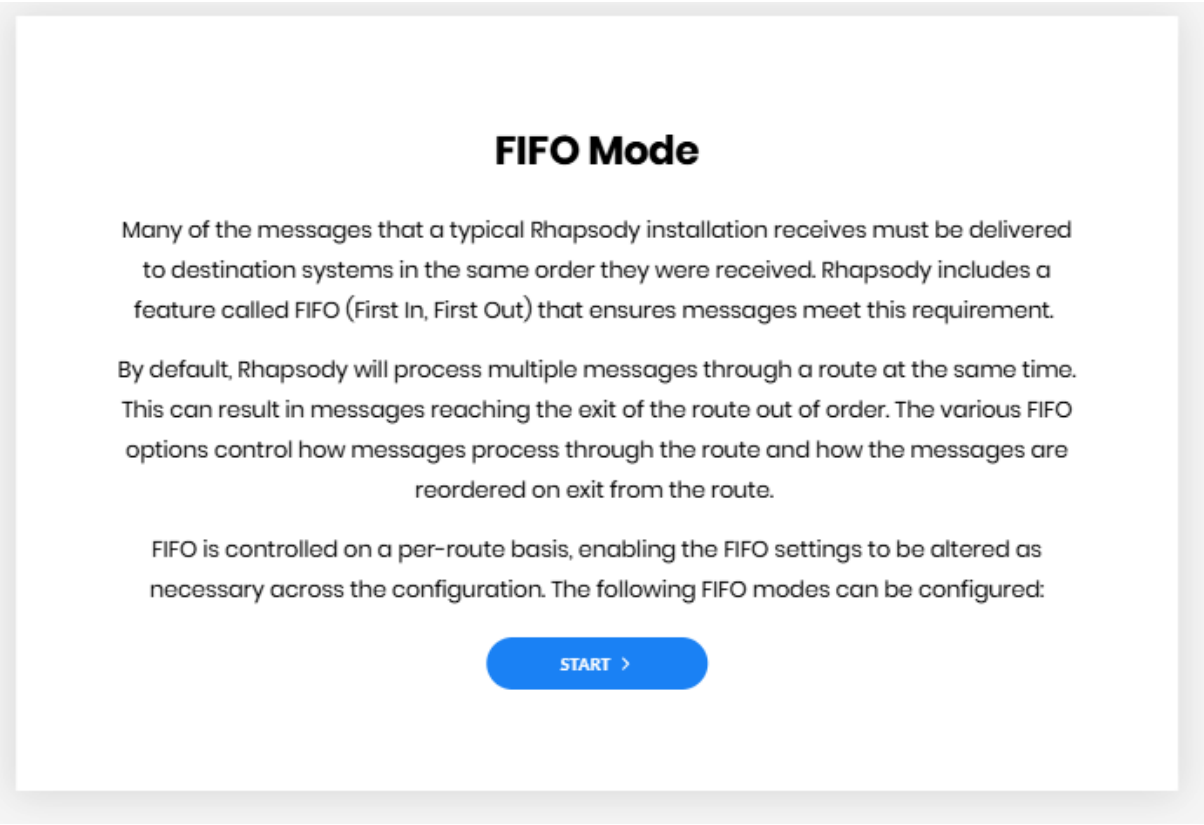


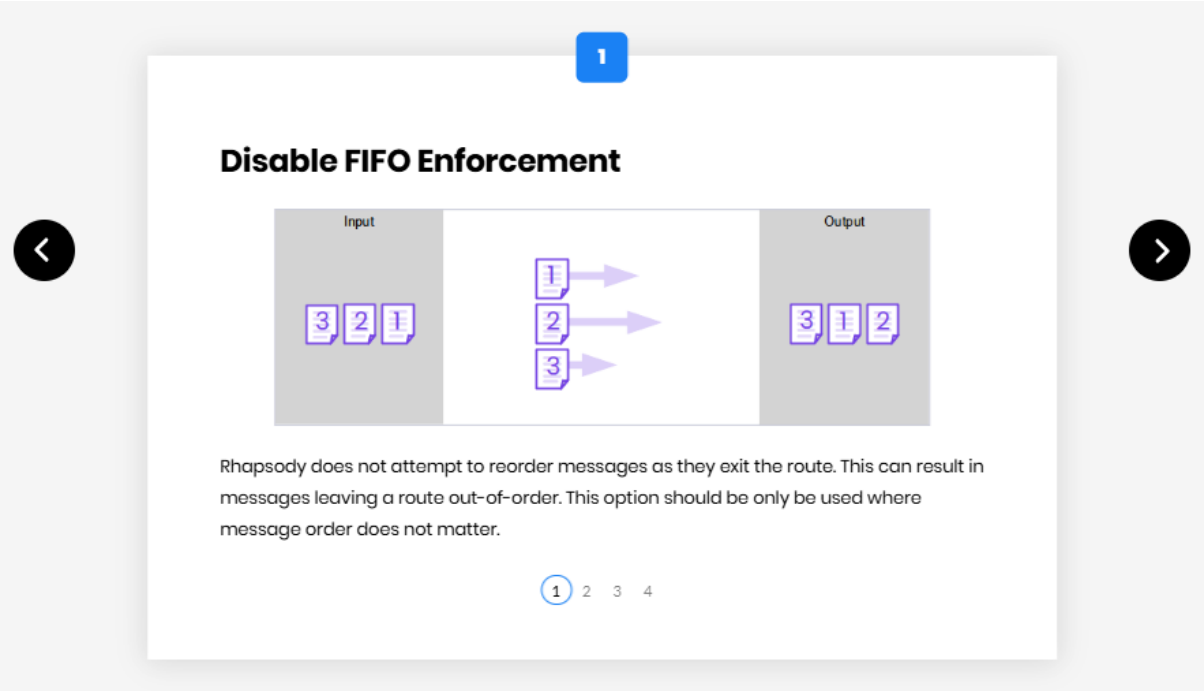
Name
A route's name should describe what processing it performs. Spaces are recommended rather than underscores to improve the name's readability. The name cannot include the following special characters: {}[]<>:;$\/*?"|.

Route Priority
From the drop-down, set the priority that Rhapsody will process messages in this route compared with other routes on the same engine. The **Realtime** setting is intended for online message processing where messages are expected to be sent and received immediately as they become available.

Startup State
This determines if the route is automatically started when the engine is started. Rhapsody normally waits for all routes and communication points at a given startup level to start before progressing to the next level. A component will be ignored if it

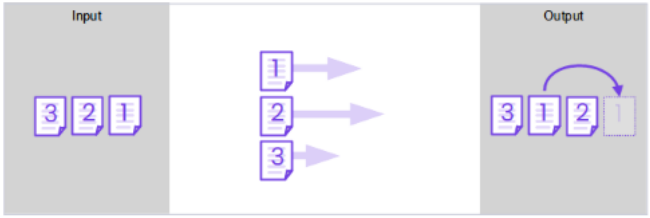
cannot be started to ensure it does not block others from starting. When you create new routes, you will normally want to change the **Startup State** to ensure your configuration starts when the engine starts. The **Start only if previously running** checkbox is only available when the **Startup State** is **Startup First** or **Startup Last**. If selected, Rhapsody remembers the route's startup state even following an engine restart.





2

Enforce FIFO by Route Input Order



Messages are reordered when exiting the route to match the input order. This ensures that messages are output in the same order as they were received.

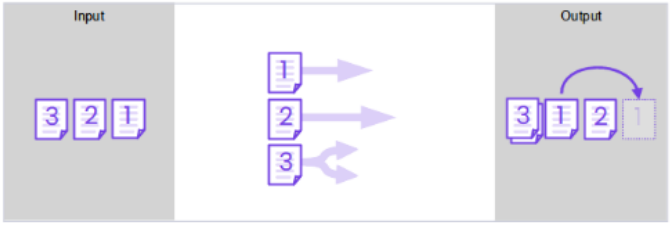
This option does not guarantee that messages will be processed in the same order by the filters within a route. If a message is delayed while processing through a filter, messages that reach the route output before that message will be held until the delayed message arrives and is sent.

This is Rhapsody's default option.

1 2 3 4

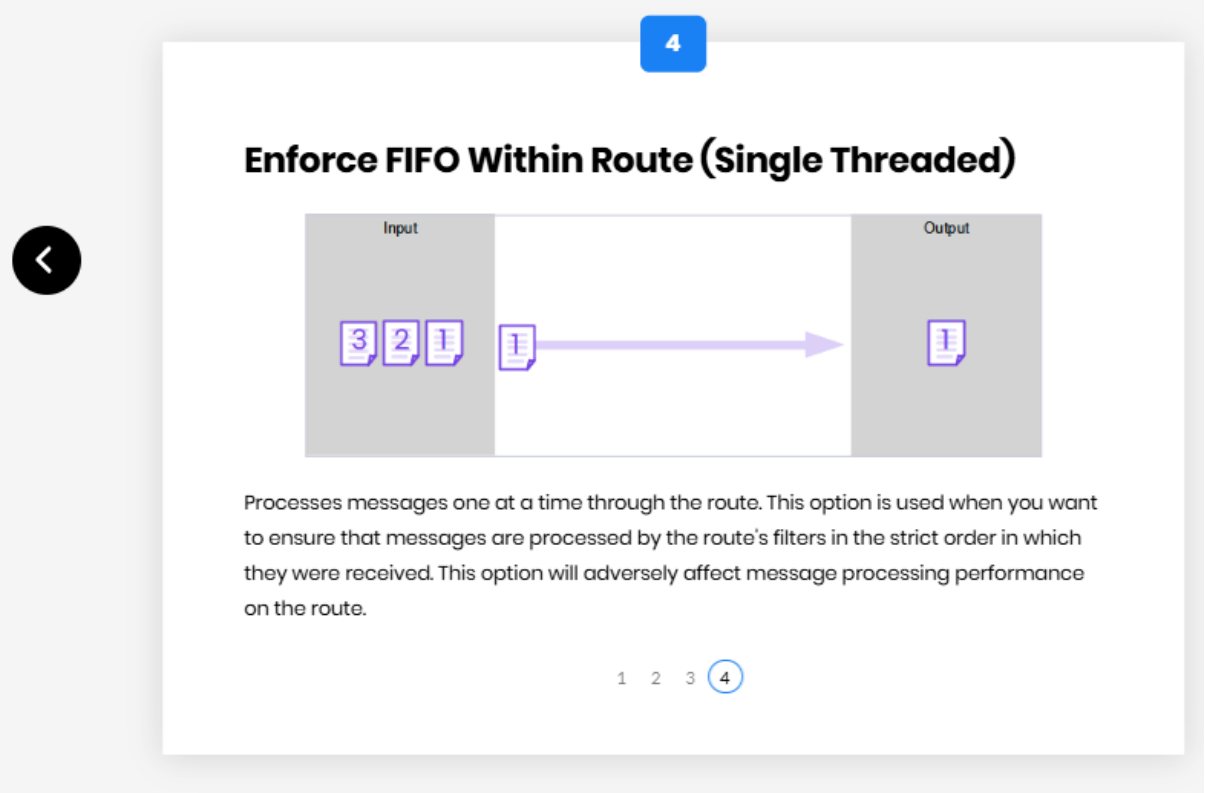
3

Enable FIFO by Absolute Message Order (De-Batching)



Functions in a similar manner to the Enforce FIFO by Route Input Order option. When a message is de-batched on the route, this option ensures that messages exit the route in the same order that they appear in the batch.

1 2 3 4



Stop the route when a message is sent to the Error Queue
When selected this will stop a route that sends a message to the **Error Queue**.

The Route Message Definition

Adding a message definition to a route allows Rhapsody to interpret the contents of matching messages during processing. Associated definitions are listed as tabs at the top of the route's **Properties** dialog.

Select **Add Definition** to associate a new definition with the route. Right-click a **Definition** tab to remove it from the route. You should never need to associate a Mapper Definition (.mdf) with a route.

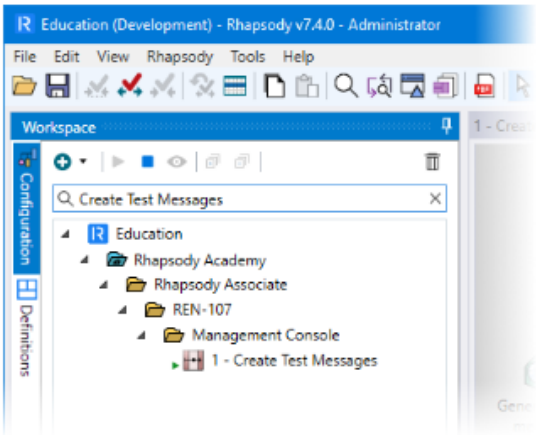
Lesson 3 of 6

Finding and Viewing Components

Rhapsody includes two methods to search for components in your configuration: **Filter Workspace** and **Find Components**.

Filter Workspace

The search box at the top of the **Workspace** lets you quickly filter the components by name; results are filtered as you type.



Find Components

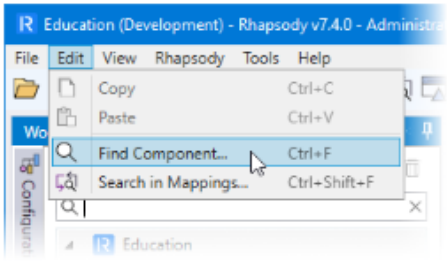
Where you need more control over your search, or you need to search the configuration of the components on the engine, **Find Components** can be used.

To perform a search:

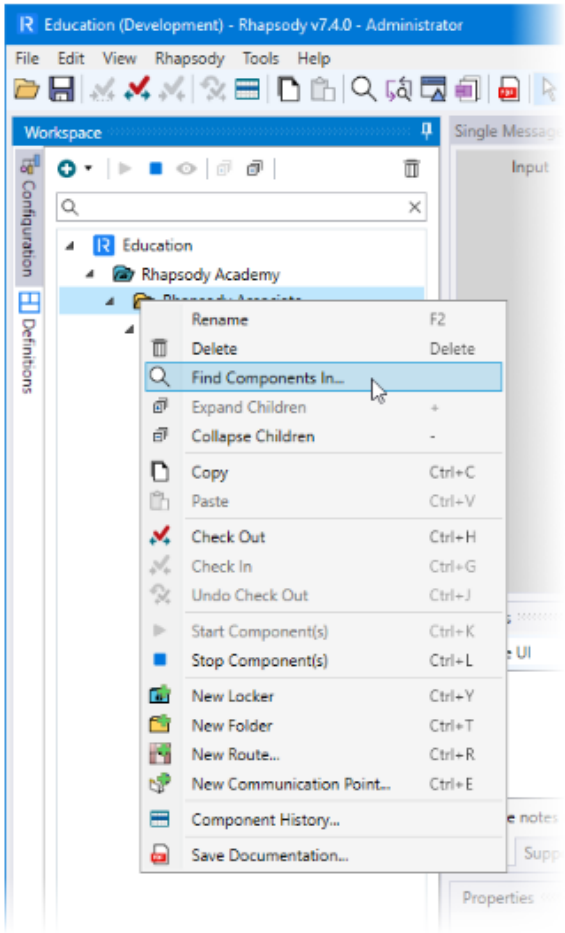
1. 1

Launch the search.

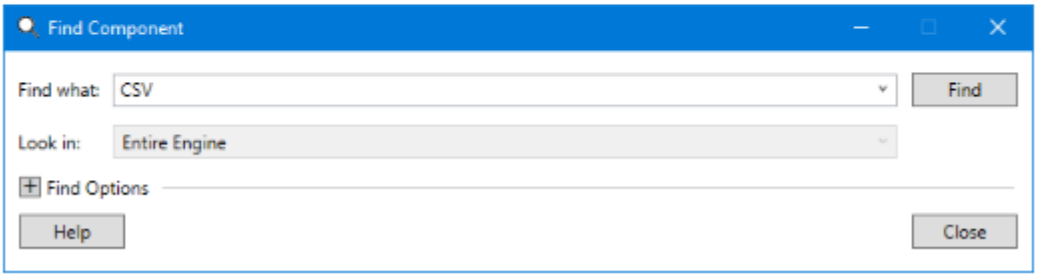
From the **Edit** menu, select **Find Component**.



You can search for components in a specific part of your configuration by right-clicking a Locker or Folder and selecting **Find Components In**.



2 In the **Find what** field, type the full or partial name of the component, then select **Find**.

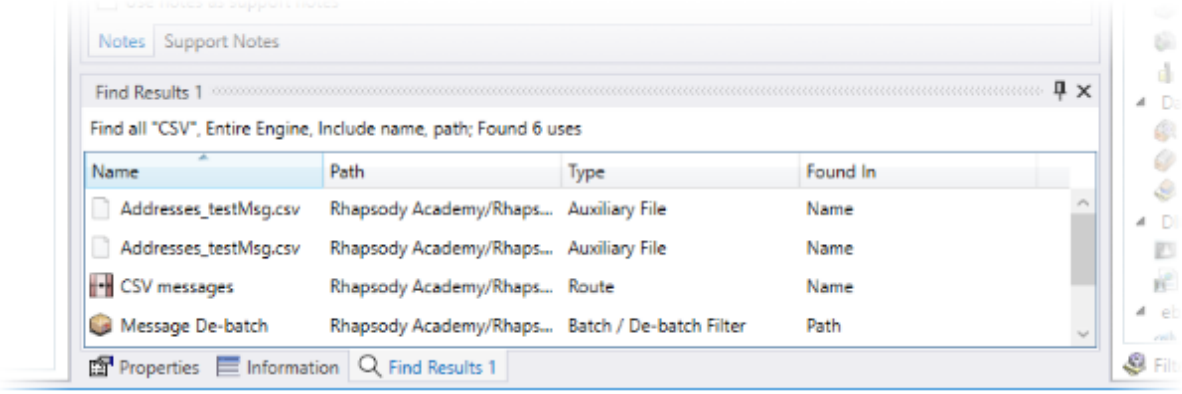


Additional search options are available by expanding **Find Options**. These let you control how the search functions and what component types will be included in the search. The options let you choose between two results tabs (**Find Results 1** and **Find Results 2**). This allows the results of two searches to be displayed at the same time.

When you reopen the **Find Components** dialog, the most recently used search criteria and options will be recalled.

Results

Results are displayed in the **Information** panel at the bottom of the window.



By default, results are sorted by **Name**. When a filter is returned by the search, its route is displayed in the right-hand column. Columns can be sorted.

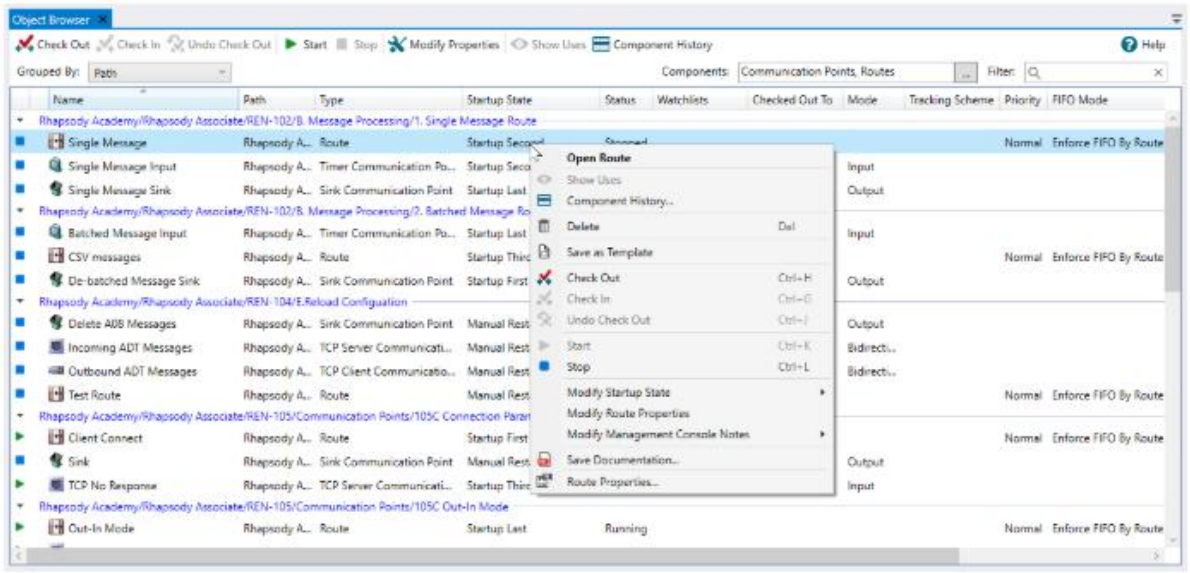
From the results, double-click a communication point or filter to open its **Properties** dialog. Double-click a route to open the route for editing.

Object Browser

Select **Object Browser** from the **View** menu or the icon on the IDE toolbar to open the **Object Browser** window. This window provides a single view of all the routes and

communication points on the engine. This is useful when reviewing or changing common settings (such as startup state) for components in the configuration.

The columns can be sorted or reordered. You can select multiple components and use the right-click menu to perform additional actions, such as delete, check in or out, start or stop.



Grouped by options

The list can be grouped by any of the columns. Select from the **Grouped By** list at the top of the window. The list is displayed ungrouped in alphabetical order by **Name**.

Filter

Objects in a configuration can be filtered using the **Components** and **Filter** options at the top of the window.

Lesson 4 of 6

Exercise - Route Building

In this exercise, we'll build the following two routes to process notification test messages that have been generated at various locations around a hospital:

- Unzip and Separate Route
- Search and Replace Route

Resources

This exercise uses messages in the *.\Associate Files and Definitions\108 - Building a Route* folder in the resources Zip file available at the start of the course.

There are three message types, each identified by their filename suffix: *.alert*, *.alarm* and *.cancel* contained inside a single Zip file.

Test Message Content

The content of a sample test message is shown below:

```
<event>
  <station>001</station>
  <type>Alarm</type>
  <action>unknown</action>
</event>
```

Messages will be processed in the following ways:

- The value associated with the station field will be a three-digit number. A value of '001' indicates that the message has originated from the testing station and will be archived. If this field has any other value, the message will be sent for further processing.
- The value associated with the type field will be one of three options: **Alarm**, **Alert** or **Cancel**. This value matches the message's filename suffix (extension).
- If the message did not originate from the testing station, further processing on the route will update the value of its action field.

Each of the test messages in the compressed archive is numbered with a filename suffix that matches its type. The full name of the first test message, for example, is: **test001.alarm**.

ⓘ When a real-world solution's configuration spans more than one route, configuring a TCP connection to join them is not the best option. The preferred methods are discussed in the *Route Interconnectivity* lesson.

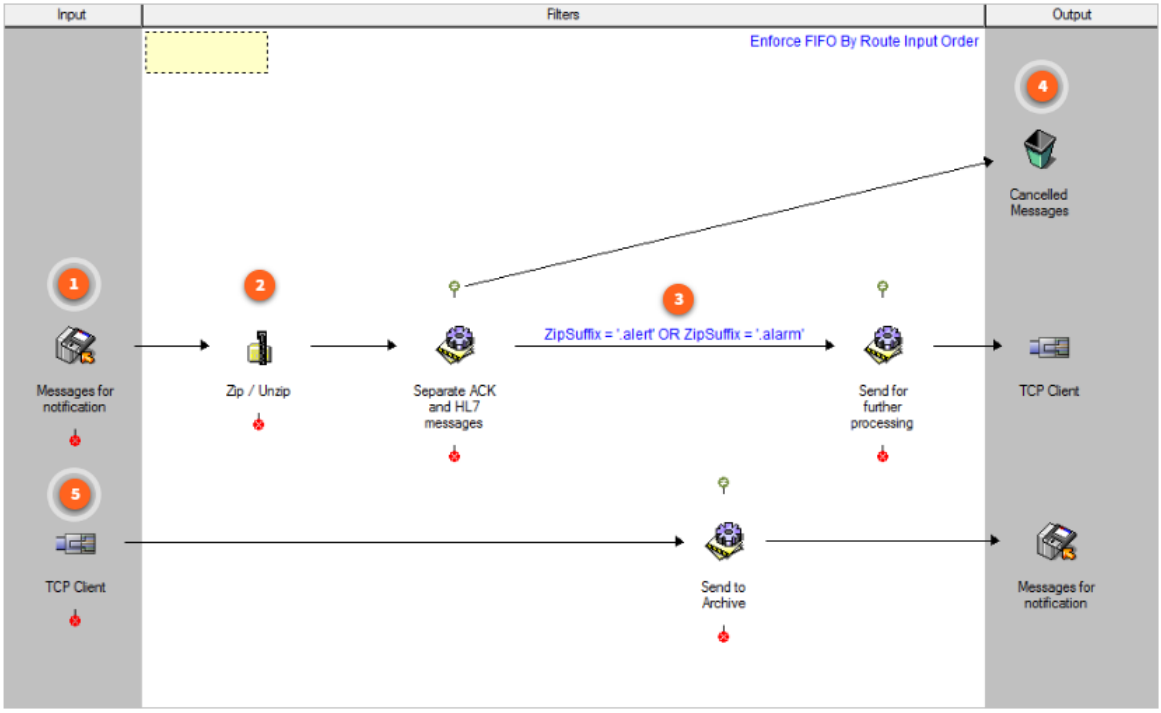
Build the Unzip and Separate route

The **Unzip and Separate** route can be summarized as follows:

1. The zipped file containing the test messages is delivered to the Input folder of a Directory communication point.
2. A Zip / Unzip filter decompresses the input file into its component messages.
3. Messages are further separated on the basis of their filename extension:

- If the extension equals **.alert** or **.alarm**, the message is sent to a second system implemented in the **Search and Replace** route via a TCP connection for further processing.
- If the extension has any other value, it is sent to a Sink communication point to be discarded.

To build the route, click each number on the screenshot and use the information provided to create each component on the route. Values formatted as a variable $(...) should be replaced with an appropriate variable in your configuration. Any properties not listed should be left at their default values.



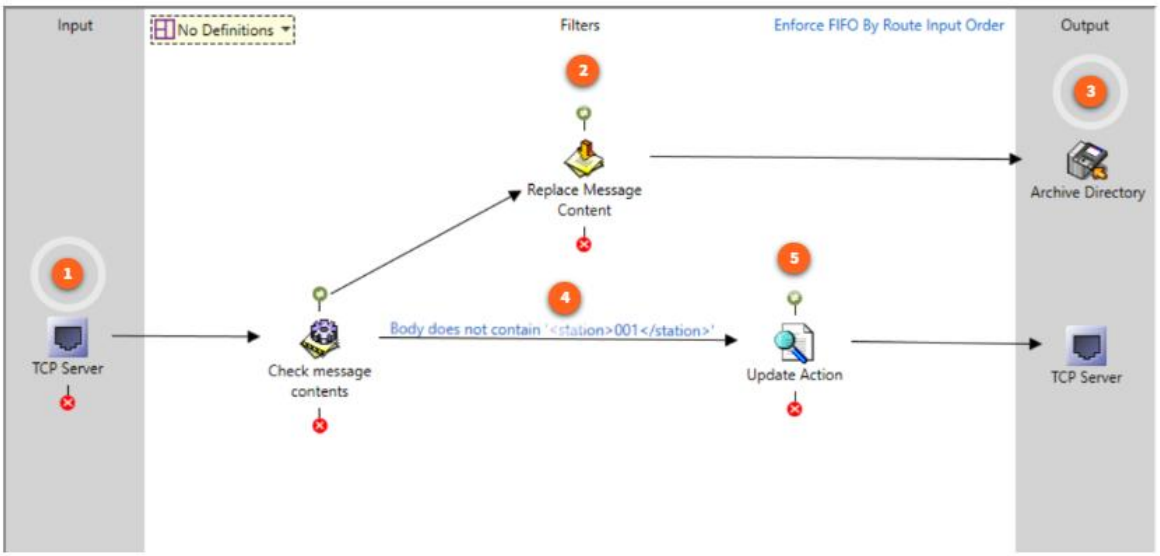
Build the Search and Replace route

The **Search and Replace** route receives filtered messages via a TCP connection. The value of the station field in each message is then checked for the following:

- Messages with a station value of **001** are sent to a Content Population filter where their content is replaced with a standard text message before being sent to a Directory communication point for archiving. In this scenario, such messages originate from a testing station and do not require additional action.
- Messages with a station value that is anything else are passed to the Update Action filter. This component is configured to update the value in the message's **Action** field from **unknown** to **notify**.

To build the route, click each number on the screenshot and use the information provided to create each component on the route. Values formatted as a

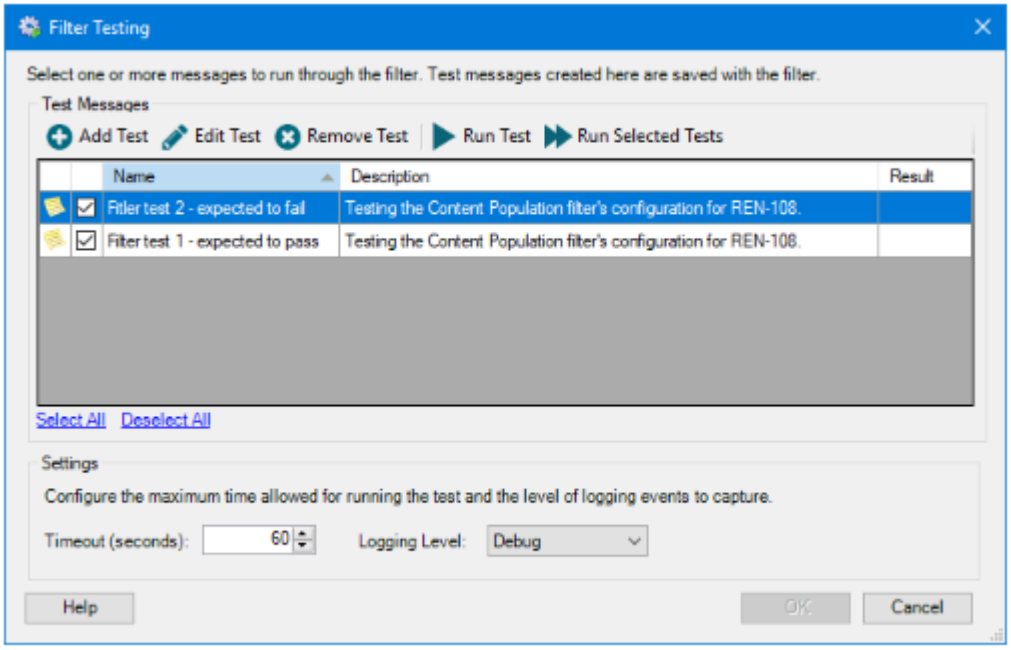
variable $(...) should be replaced with an appropriate variable in your configuration. Any properties not listed should be left at their default values.



Test the Content Population Filter

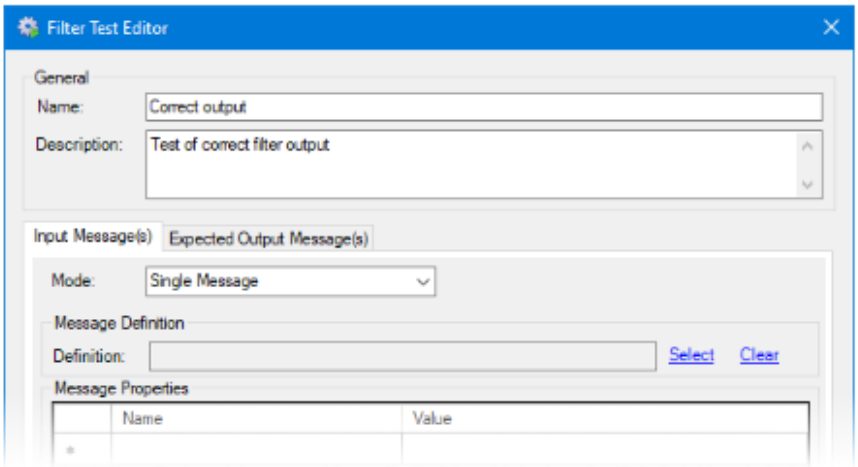
It is best practice to test components in isolation before being assembled into the final solution. This can significantly reduce its overall testing effort. We're now going to test the **Content Population** filter created for this solution, including the validation of the test results. The purpose is to demonstrate that the messages processed by this filter match in every respect the expected structure and content of the output messages.

Begin by opening the **Content Population** filter's **Properties** dialog **Configuration** tab, then select the **Test Filter Configuration** link to open the **Filter Testing** dialog. The following dialog shows the two test messages that you will create for this exercise. This window was discussed in detail in the "Filters" module.



Add test

Select the **Add Test** icon to open the **Test Editor** dialog. As can be seen above, and following best practice, multiple tests will be created for this exercise. One that checks an expected message, and one that tests an error condition. Type the **Name** and **Description** for the message expected to pass.

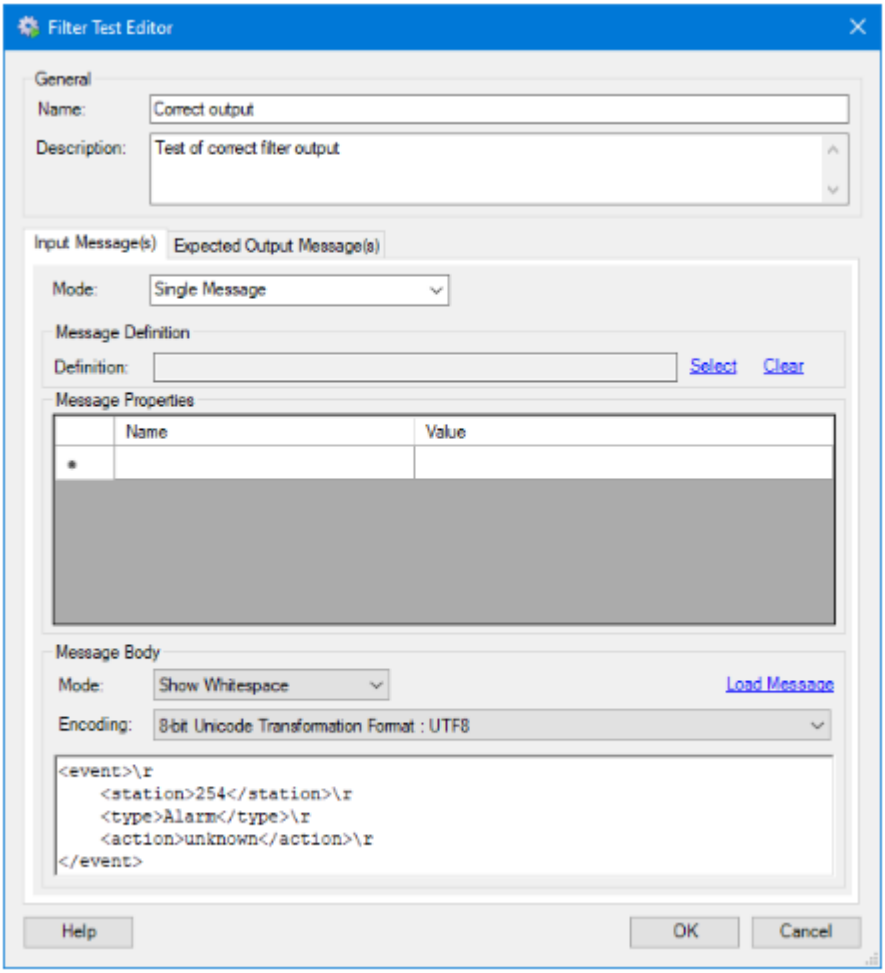


The remaining options depend on the **Input Message(s)** or **Expected Output Message(s)** tab selections.

Input Message(s) tab

The **Input Message(s)** tab in the filter's Test Editor configures the message that will be tsted by the filter.

The following screenshot shows the configuration of the **Input Message(s)** tab for the *Correct output* test message. Make the selections and enter the information as shown below:



1. **Definition**
   A message definition is not required for this test as the filter does not rely on the an existing message definition being associated with the message.

2. **Message Properties**
   There are no message properties associated with the *Filter Test 1* test message.

3. **Message Body**
   Select the **Load Message** link to load the *FilterTest_InputMessage.alarm* input message. Leave the **Message Body** options at their default settings:

   1. **Mode**
      Show Whitespace (default) uses Java escape characters to represent

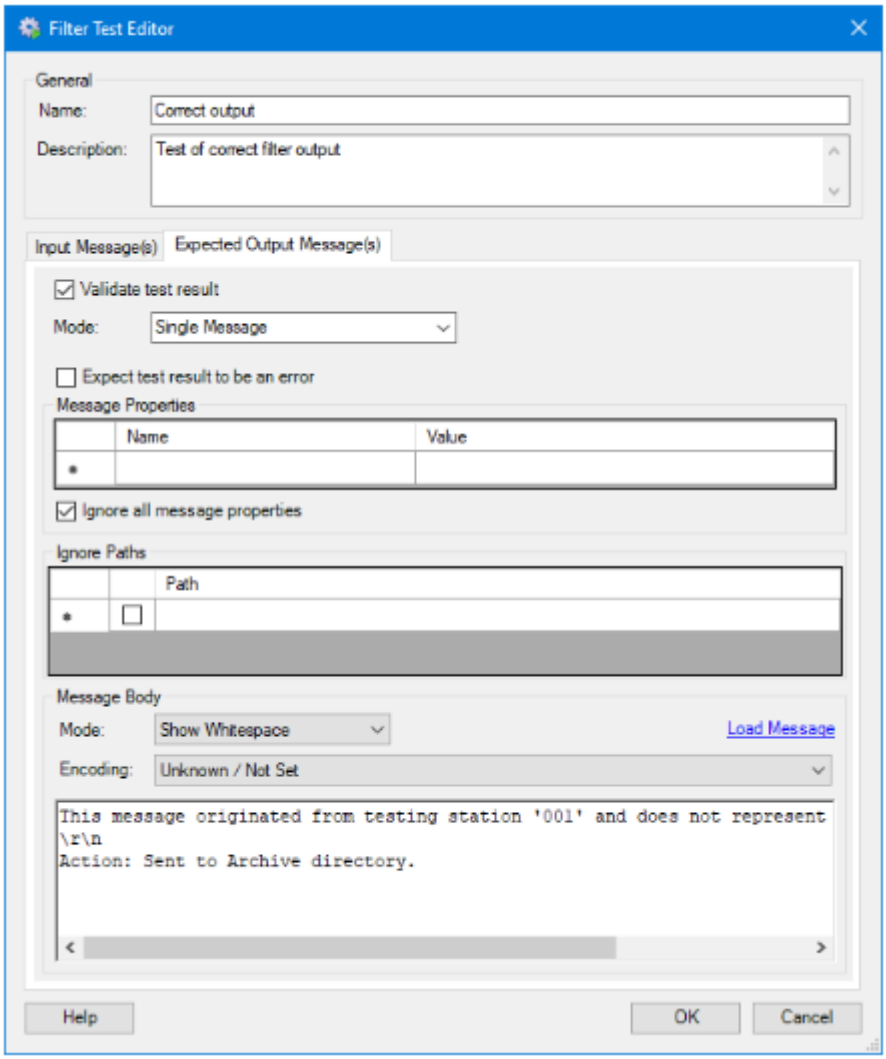
non-printable characters. **Raw Hex** mode enables direct editing of the hexadecimal characters making up the binary message.

2. **Encoding**
   8-bit Unicode Transformation Format : UTF8 is the message's default character encoding. If nothing is selected, message encoding will not be applied to incoming messages.

Expected Output Message(s) tab

The **Expected Output Message(s)** tab configures the range of messages that are expected to be produced following the completion of the test.

The following screenshot shows the configuration of the **Expected Output Message(s)** tab for *Filter test 1*. Make the selections and enter the information as shown below:



1. **Validate test result**
   Compare the test's actual output message with the message loaded onto this screen.

2. **Expect test result to be an error**
   If set indicates that the message will cause an error in the filter. As we expect our test message to process without error, this option should be left unchecked.

3. **Message Properties**
   Manually add or edit the message properties used during output message validation.

   It does not matter whether the **Ignore all message properties** checkbox is selected as we didn't specify any properties as part of the input message and the filter is not going to create any properties.

4. **Ignore Paths**
   Add or edit the message paths whose contents are to be ignored during output message validation.

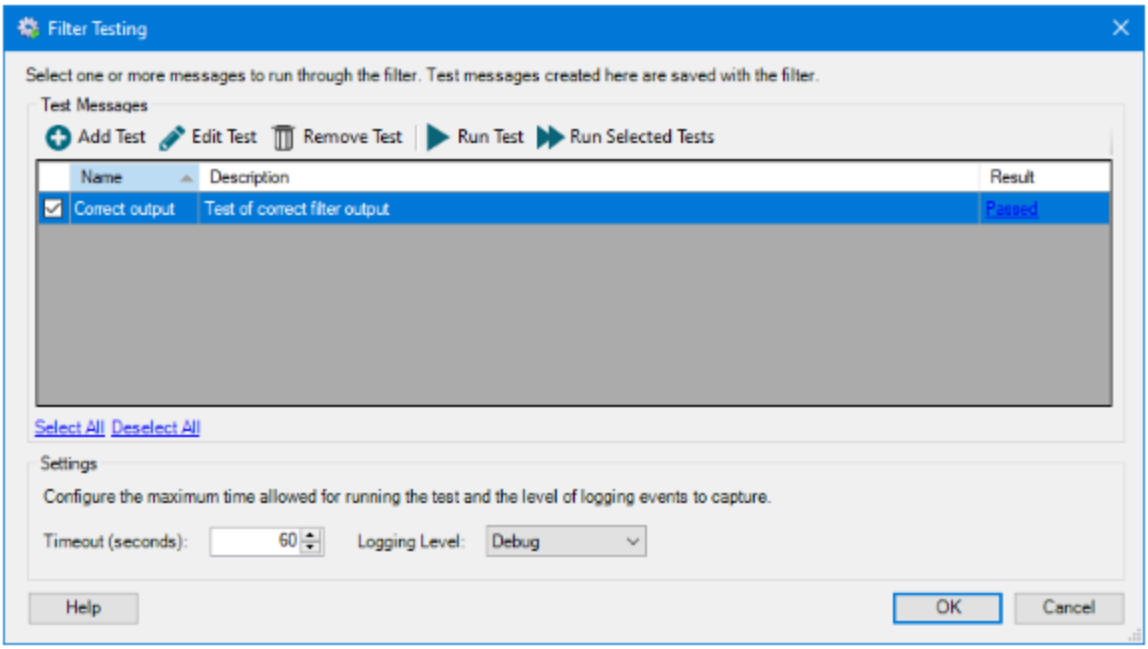
   This option is typically used for messages in HL7 format which have an associated message definition and for which fields such as DateTimeOfMessage and MessageControlID fields must be ignored when validating the output message.

5. **Message Body**
   Select the **Load Message** link to load the *FilterTest_Expected OutputMessage - PASS.alarm* test message.

When you run the test, Rhapsody uses the **Content Population** filter's configuration to process the input message identified on the previous tab. The message resulting from this processing is compared with the expected message body identified above; if they match then the result is validated, and the filter's configuration is assumed to be correct.

Running the Test - PASS

When the configuration of the test's input and expected output messages has been completed, the test can be run. If only one test message is involved, selecting it and selecting **Run Test** will run the test. If multiple test messages are available and you want to run the tests simultaneously, select their accompanying checkboxes and select **Run Selected Tests**.



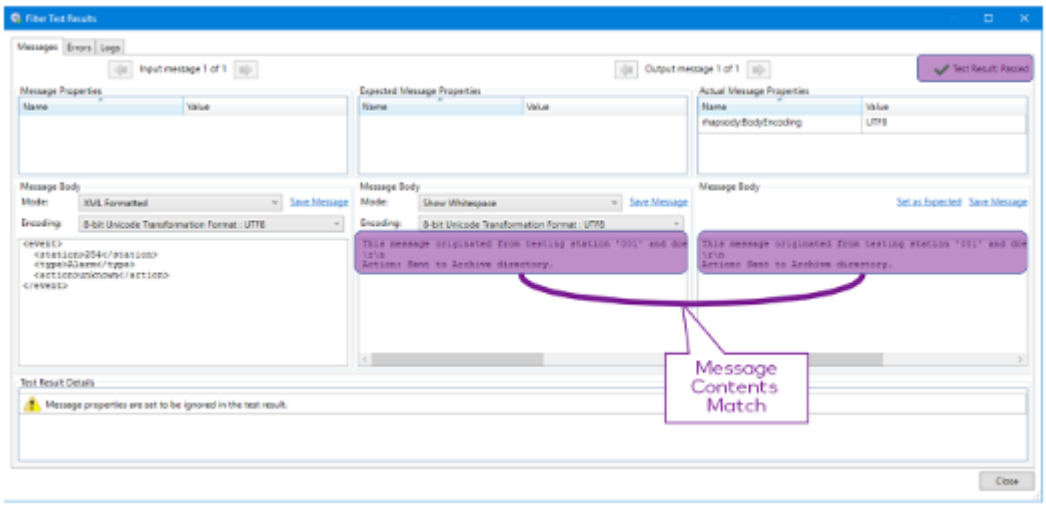
Running the test executes the **Content Population** filter's configuration against the input test message(s) and displays the outcome in the rightmost column. The possible options are:

- **Passed**
  The test has executed successfully with the result passing validation. In other words, all message properties and content for the actual and expected messages matched.
- **Failed**
  The test has executed successfully but with the result failing validation. In other words, at least one message property and/or the content of the actual and expected messages did not match.
- **Executed**
  The test has executed successfully. Output messages, however, were not validated.
- **Error**
  One or more fatal errors have occurred while running the test, preventing it from successfully executing.
- **Skipped**
  The test did not run, usually because no input messages had been selected when testing in **Message Collection** mode.
- **Invalid**
  There are two causes of an invalid result:

- The filter does not support filter testing. Examples include the **Duplicate Message Detection** and **Hold Queue** filters.
- The filter is connecting to a non-standard database using custom database drivers. This could apply to the **Database Look Up** and **Database Message Extraction** filters.

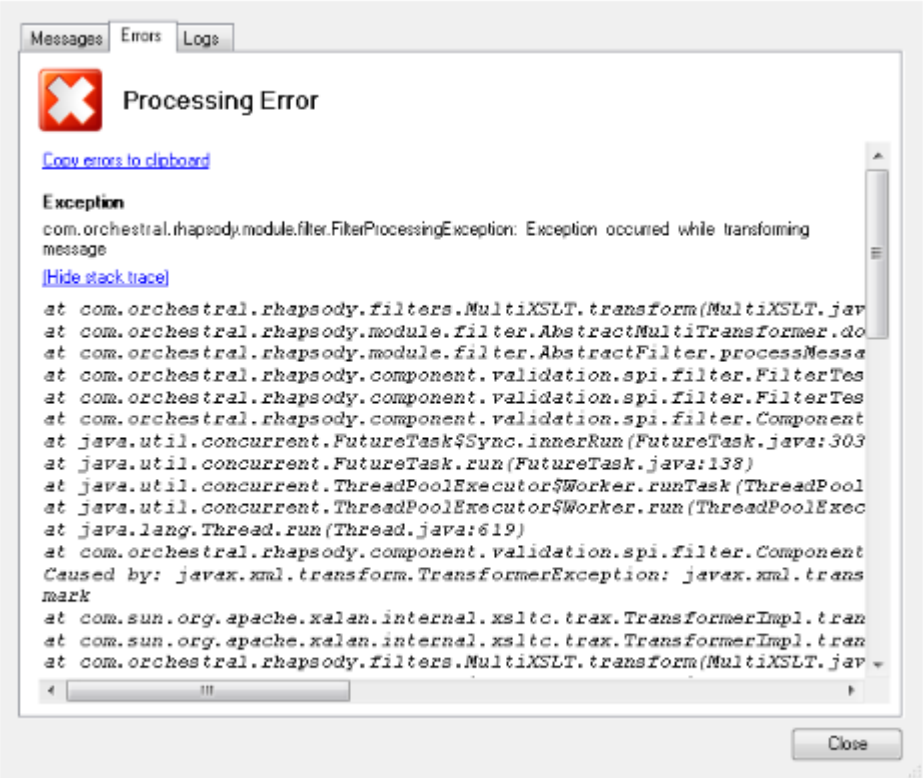
Test result: Passed

Select the **Passed** link to show the **Filter Test Results** window for a validated test result and confirm that the result in your test matches the screenshot below.



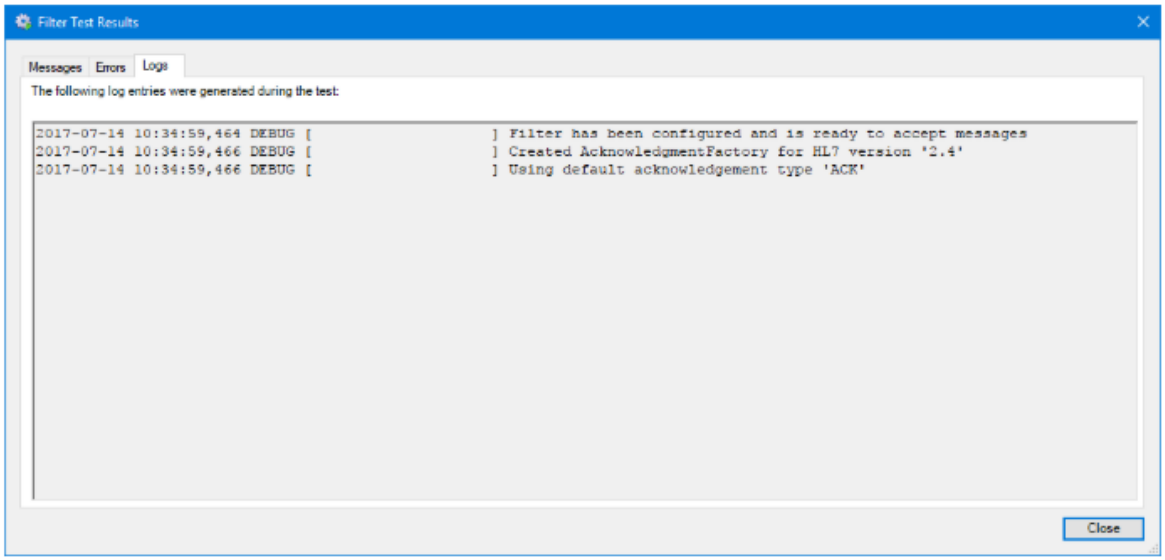
Errors

Selecting the test result's **Errors** tab identifies any errors that may have occurred during the processing of the test message by a filter or conditional connector. Selecting the **Copy errors to clipboard** link can be used to send this information to a second administrator. Checking **stack trace** will often help identify the source of this error, which can then be corrected before re-running the test.



Logs

Selecting the **Logs** tab displays the entries saved to the Rhapsody log at the time of the test's completion. The level of detail to include in the log is configured on the **Filter** or **Conditional Connector Testing** windows. Trace adds the most information. Debug is the default. Fatal includes all actions that prevented the filter from working.



Only those log entries generated by the filter or conditional connector during testing are displayed on this window. If other Rhapsody services are called during test execution, their log entries will not be displayed but can be seen in the main Rhapsody log. The name of the sub-logger creating these entries will appear between the square brackets in the above image.

Running the Test - FAIL

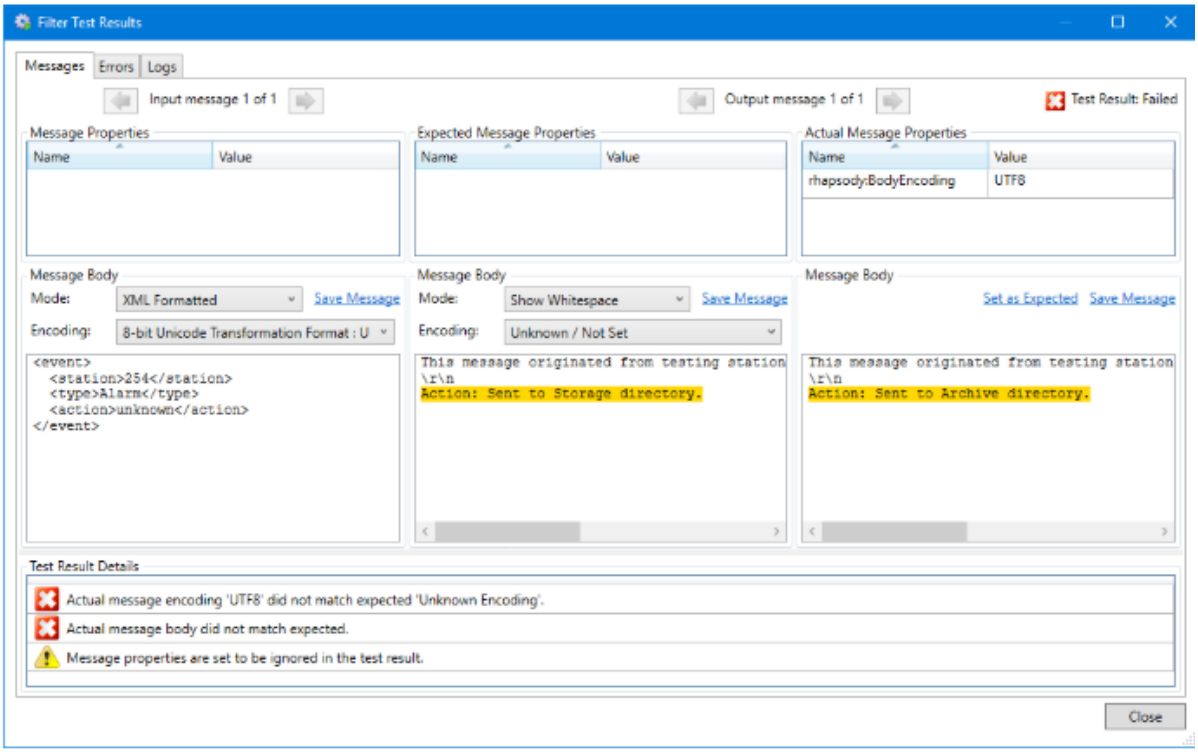
There are two ways in which a message might fail validation:

- The actual and expected message properties do not match.
- The actual and expected message's body content do not match.

1. Follow the steps above to add the second test message with a name and description that identifies it as one expected to fail validation.
2. Load the following messages as the **Message Body**:
    1. **Input Message(s)**: *FilterTest_InputMessage.alarm*
       This is the same input message as that used in the previous test.
    2. **Expected Output Message(s)**: *FilterTest_ExpectedOutputMessage - FAIL.alarm*
       This is a different expected output message from that used in the previous test.
3. All other settings can be left unchanged, including the selection of the **Ignore all message properties** checkbox on the **Expected Output Message(s)** tab. Return to the **Filter Testing** dialog and run the second test.

The test result

Running the second test should result in the appearance of a **Failed** link on the **Filter Testing** dialog. Selecting this link displays the **Filter Test Results** screen shown below, with the cause of the failure clearly identified.



Fixing the errors

The expected message body's contents could be edited to remove the mismatch with the actual message body. Alternatively, selecting the **Set as Expected** link will set the expected message's body content to be the same as the actual content. When this is done, and provided the same test message was reused, the test result will show as **Passed** as the two message contents now match.

✎ When creating filter tests, you want any test with an expected output to pass. This makes it easy to review all the tests in a configuration and know what is working correctly.

Testing the Solution

After the configuration for the two routes has been completed, check in and start all components. Copy the *Test Messages.zip* file from the exercise resources to the Input folder of the Directory communication point in the **Unzip and Separate** route.

Model Answer

A completed solution for this exercise, can be found in the resources directory for this exercise. It can be used for comparison with your own work.

The compressed test message contains twelve individual messages. These messages will be separated and distributed as described below following processing by the routes in this exercise:

- **Cancelled Messages** Sink communication point - three messages.
    - The **ZipSuffix** message property associated with these files will be .cancel.
    - The counts can be confirmed by comparing the Before and After message counts for this communication point in the Rhapsody Management Console.
    - The property's association can be confirmed by viewing the **Output Message Archive** for any message sent to this communication point.
- **Archive Directory** Directory communication point, Output folder - three messages.
    - The content of these messages should match the content of the Body Template File specified in the configuration of the **Content Population** filter in the **Search and Replace** route.
- **Messages for Notification** Directory communication point, Output folder - six messages. These messages will all have:
    - a station that is not 001.
    - a type that is either *Alert* or *Alarm* (it will not be *Cancel*).
    - an action that is *notify* (it will not be *unknown*).
- *Lesson 5 of 6*

Route Interconnectivity

After a message has been received by Rhapsody, it will normally be processed by several filters before being sent to one or more outbound communication points. Sometimes the same message will be required by many outbound systems or the level of complexity of the configuration will exceed what can comfortably be added to a single route. When this occurs, the configuration can be split across multiple routes. This allows messages to be sent from one source route to one or more destination routes. This lets specific business logic be applied to match the needs of each receiving system.

In this lesson we will discuss the two most common methods of connecting routes in Rhapsody:

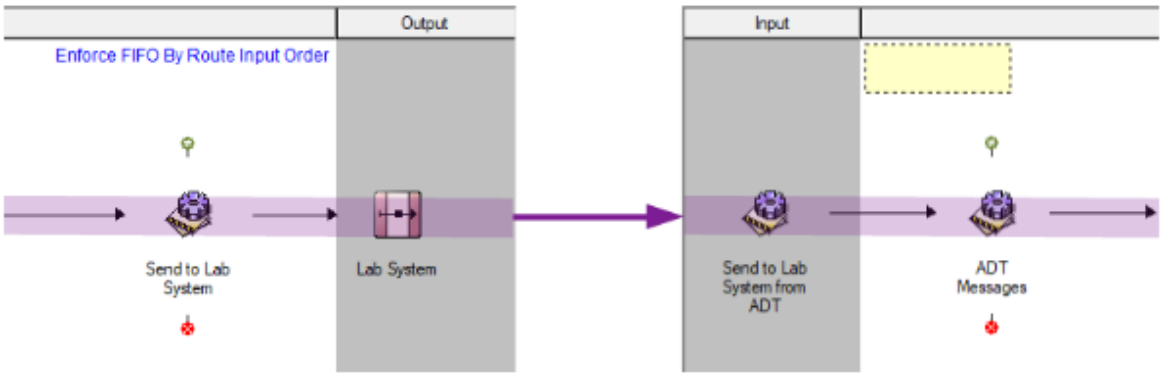
- Direct Route Connection
- Dynamic Router Communication Point

Direct Route Connection

Routes within the same locker may be connected by a Direct Route Connection. You do this by dragging the destination route into the Output section of the source route. Connections are then established in the normal manner. It is important to protect the inter-route connections when using a Direct Route Connection. This is achieved by placing No-operation filters immediately before and after the interconnect:

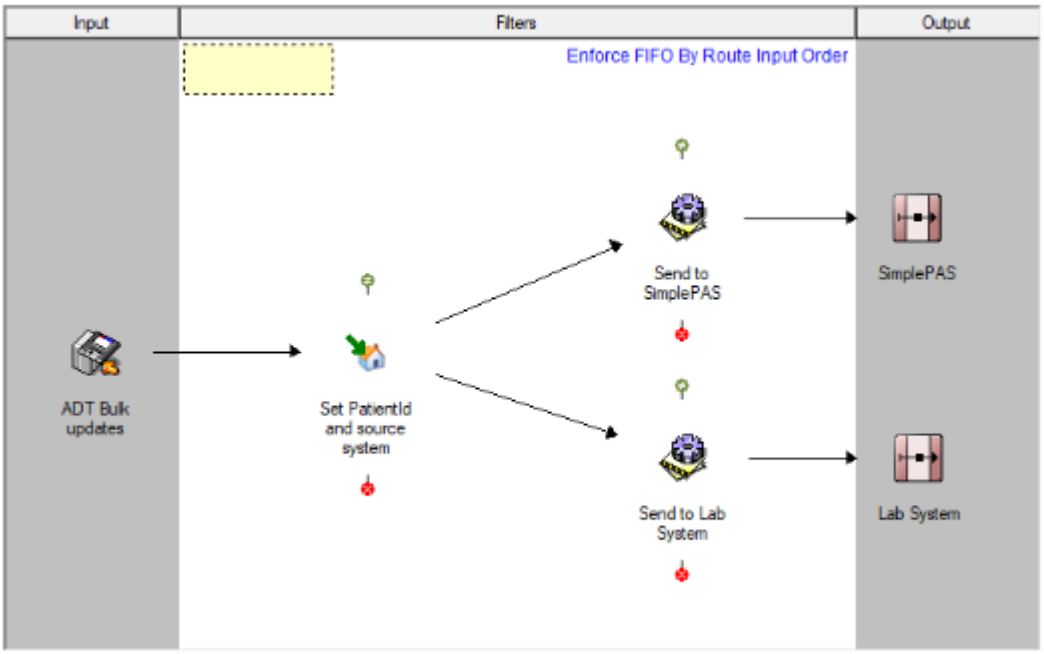
- On the upstream route, place a No-operation filter as the final filter in the path; the connection between the filter and the route in the Output frame should be a simple connection (that is, without a condition).
- On the downstream route, place a No-operation filter as the first filter in the route.

This technique is referred to as bookending a route. This will preserve and protect the connections from other changes to the routes.



Multiple routes can be added to the Output section of the workspace by dragging the next destination route to the Output section of the workspace.

There is no limit on the number of interconnected routes that can be placed on the Output section of the workspace.



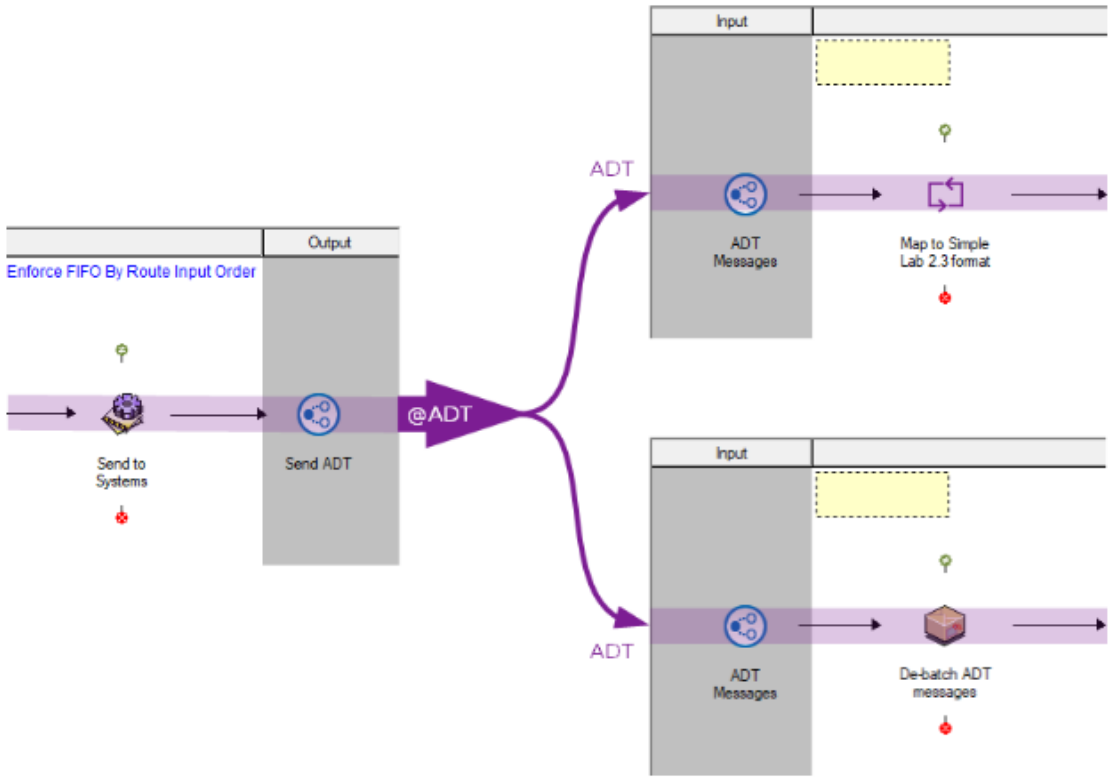
This method works well when there is a small static set of destination routes. Best practice is to use Dynamic Router communication points when there are enough connected routes to require the use of scrolling to see all connections.

Dynamic Router Communication Point

The Dynamic Router communication point provides an alternative mechanism to cross-route connectors for connecting routes together in Rhapsody. The dynamic router is the only way to transfer messages between lockers. Instead of explicitly wiring cross-route connectors between filters, a set of dynamic routers can be used to route messages statically or dynamically.

Using the dynamic router for routing messages can be described as a "Publisher-Subscriber" method of routing messages. A dynamic router set to Output mode will publish (send) messages with a particular target name. Separately, one or more dynamic routers set to Input mode can subscribe (receive) to a target name to receive messages. This allows one-to-many, many-to-one, and many-to-many connections to be created.

The dynamic router target can be either configured statically in the Output communication point properties, or can be set dynamically by using the router:Destination message property.



To prevent a dynamic router communication point in Input mode from processing messages, the route it is connected to must be stopped.

Comparison

While both interconnect methods provide a method of sending messages to connected routes, they both have advantages and disadvantages that should be considered.

Direct Route Connection

Advantages

- It is simple to drag and drop a destination route onto a source route.
- Double-clicking the route icon in the Input or Output section of the route is a direct link to the connected route.
- Message properties are preserved.
- Continual tree view in the Management Console.

Disadvantages

- When connecting to multiple routes, the source route Output section can get overcrowded and difficult to maintain.
- When migrating configurations, having dependent routes can make the migration more difficult.

Dynamic Router Communication Point

Advantages

- One Publisher Dynamic Router can send to multiple subscribing routes. This can be dynamically set at runtime or can be a static destination.
- Makes migrating the configuration less complicated.
- Message properties are preserved.
- Continual tree view in the Management Console.
- Messages can be sent across Lockers.

Disadvantages

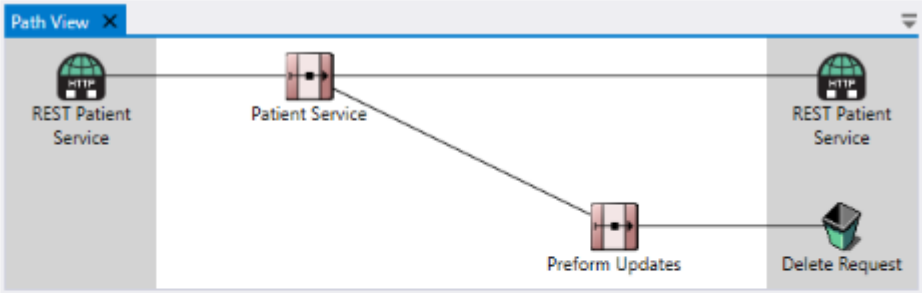
- Naming convention needs to be clear, otherwise it may not be obvious what the destination for a message sent to a dynamic router will be.
- Without careful design, it is possible to create an infinite loop between routes.

The simple drag-and-drop approach of the Direct Route Connector makes it appealing for small configurations that span two or three routes. However, when additional routes are required or where a route publishes messages to many downstream routes, the Dynamic Router becomes a much better tool.

Path View

Path view is used to illustrate how routes and communication points are connected. This is of greater use when direct route connections have been used to join two or more routes together.

Select **Path View** from the IDE **View** menu, then select a Route or Communication Point from the **Workspace**. This displays an overview of the Rhapsody configuration associated with the selected component.



ⓘ The route along with its associated communication points must be checked in before **Path View** can be used.